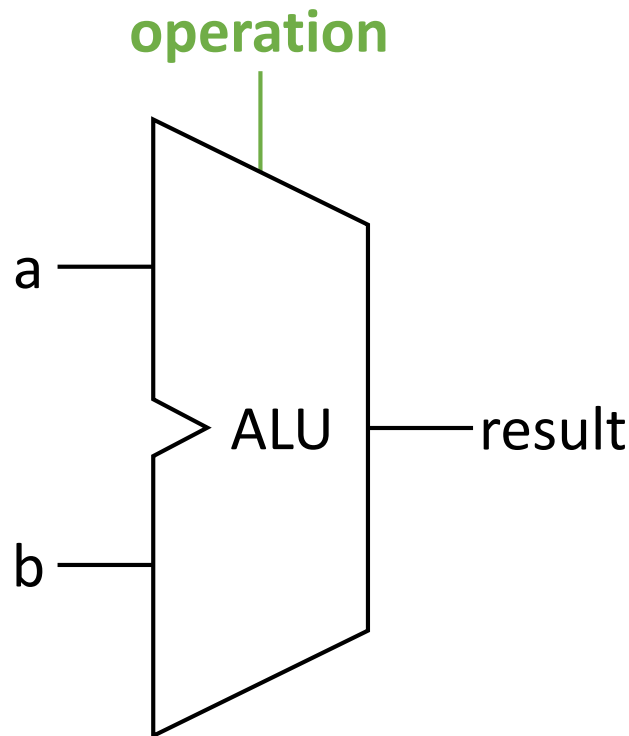


Lecture 7: Building an ALU

CMPS 221 – Computer Organization and Design

Arithmetic Logic Unit

- An **arithmetic logic unit (ALU)** is a logic block that performs arithmetic and logic operations on integers (e.g., $\&$, $|$, $+$, $-$, $<$, $==$, ...)

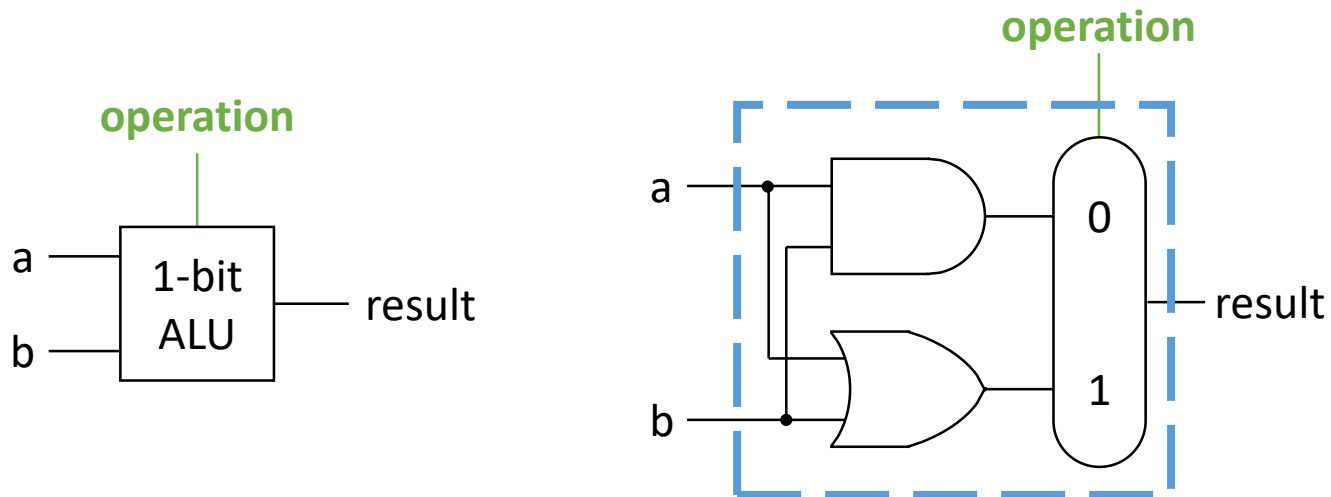


1-bit ALU

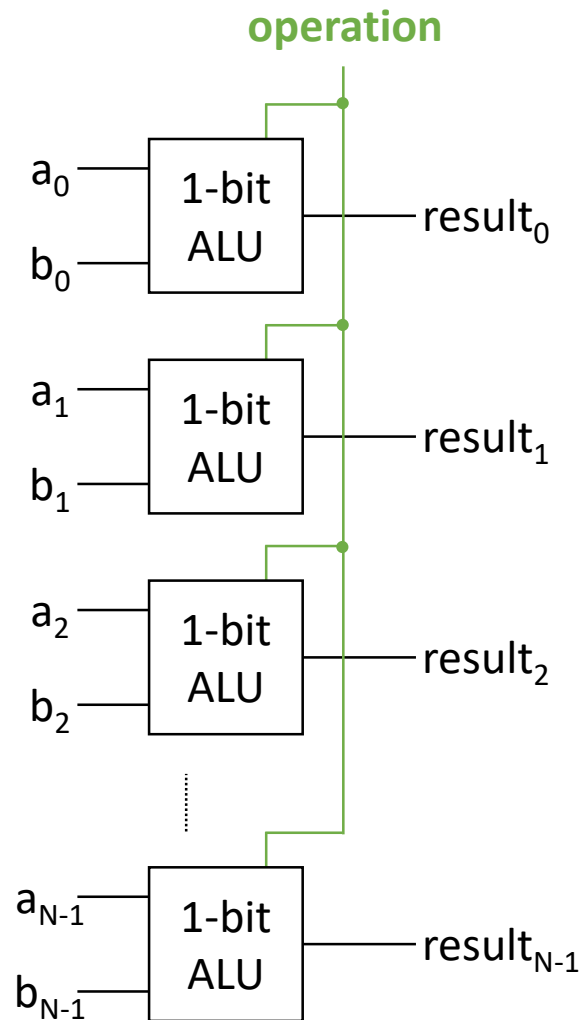
- Different architectures support primitive types with different numbers of bits
 - e.g., MIPS supports 32-bit and 64-bit primitive types
- Assume that we can construct a N -bit ALU by connecting N 1-bit ALUs
 - Start designing a 1-bit ALU

Logic Operations

- Logic operations are straightforward because they map directly to logic gates and are confined to the 1-bit ALU

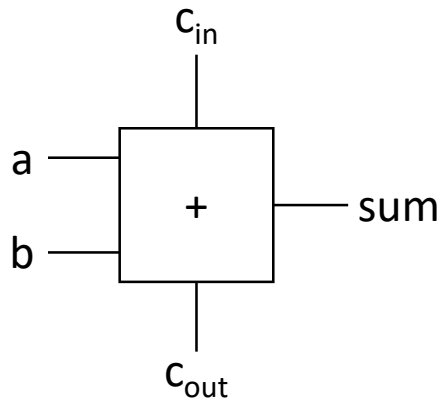


N-bit ALU

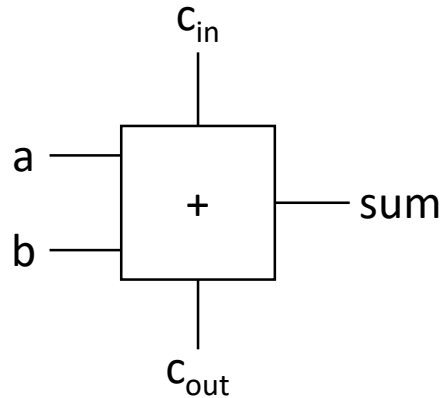


Addition

- Recall: addition may require receiving a carry bit from the previous place value and passing a carry bit to the next place value



Implementing a 1-bit Adder



Inputs			Outputs	
a	b	c _{in}	c _{out}	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

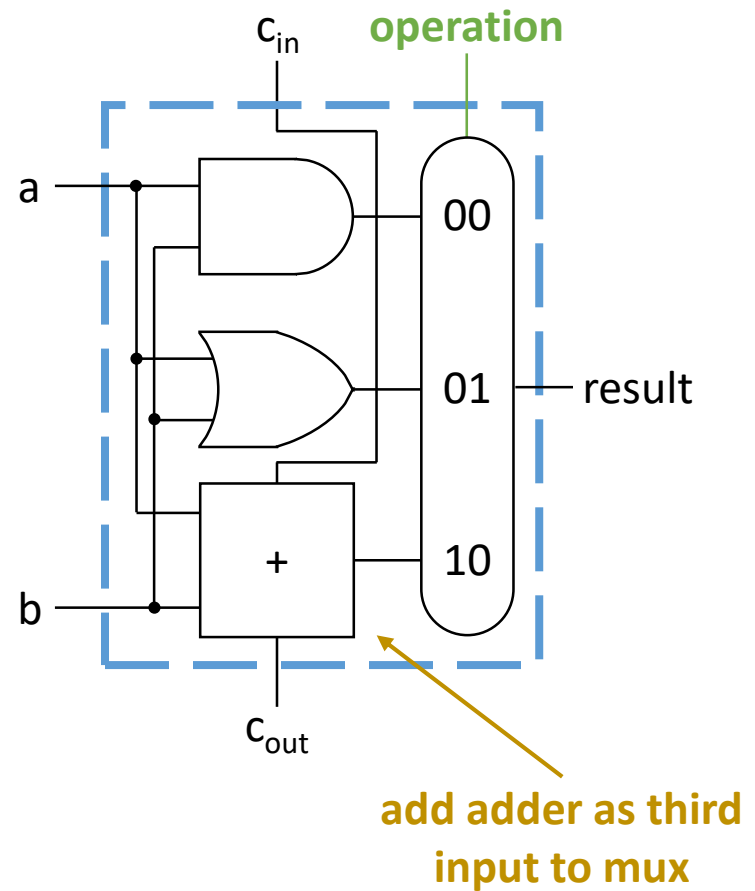
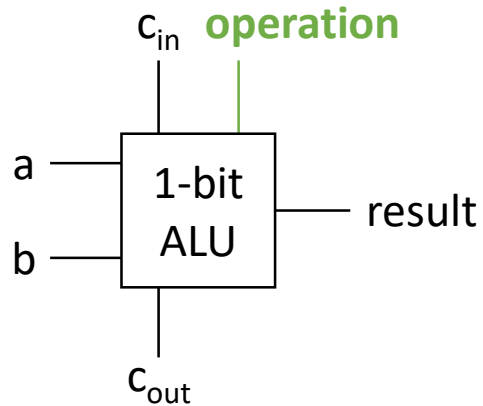
		ab			
		00	01	11	10
c _{in}	0	0	0	1	0
	1	0	1	1	1

$$c_{out} = a \cdot b + a \cdot c_{in} + b \cdot c_{in}$$

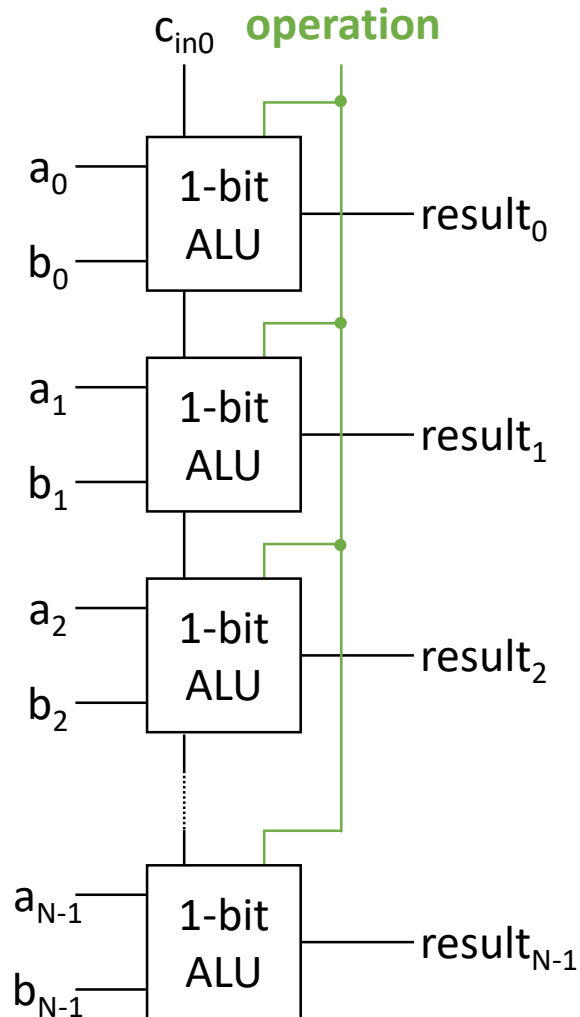
		ab			
		00	01	11	10
c _{in}	0	0	1	0	1
	1	1	0	1	0

$$\begin{aligned} \text{sum} = & a \cdot \bar{b} \cdot \bar{c}_{in} + a \cdot b \cdot c_{in} \\ & + \bar{a} \cdot b \cdot \bar{c}_{in} + \bar{a} \cdot \bar{b} \cdot c_{in} \end{aligned}$$

1-bit ALU with Addition



N-bit ALU with Addition



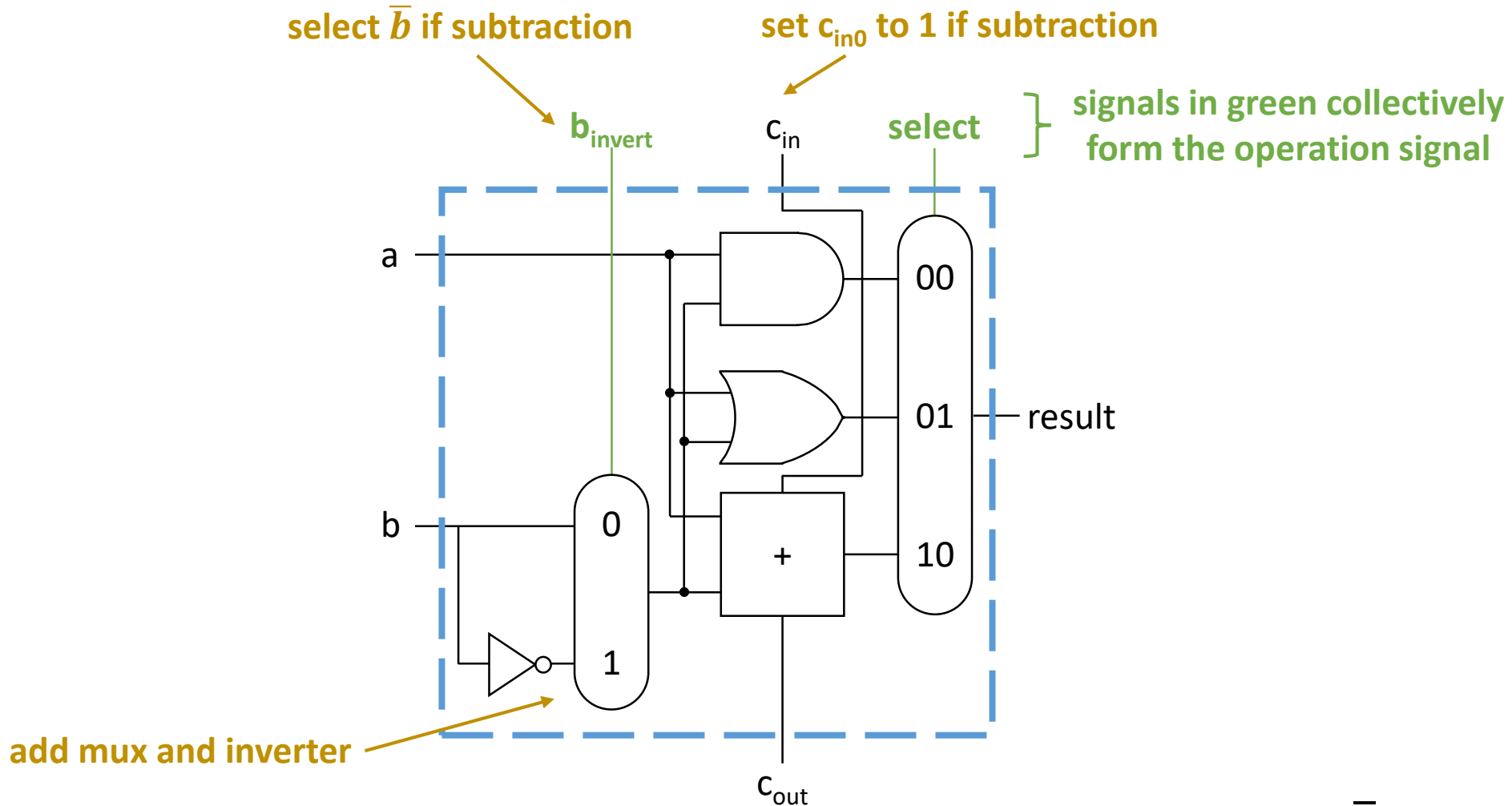
The adder created by this configuration is called a **ripple carry adder** because the carry “ripples” across the 1-bit adder

Subtraction

- Recall:
 - Subtraction can be performed by adding the two's complement
 - Two's complement can be found by inverting all bits and adding one
- To implement subtraction:

$$a - b = a + (-b) = a + \bar{b} + 1$$

1-bit ALU with Subtraction



$$a - b = a + \bar{b} + 1$$

NOR and NAND

- Some architectures may also support NOR/NAND
- Use DeMorgan's Laws

$$\text{NOR: } \overline{a + b} = \bar{a} \cdot \bar{b}$$

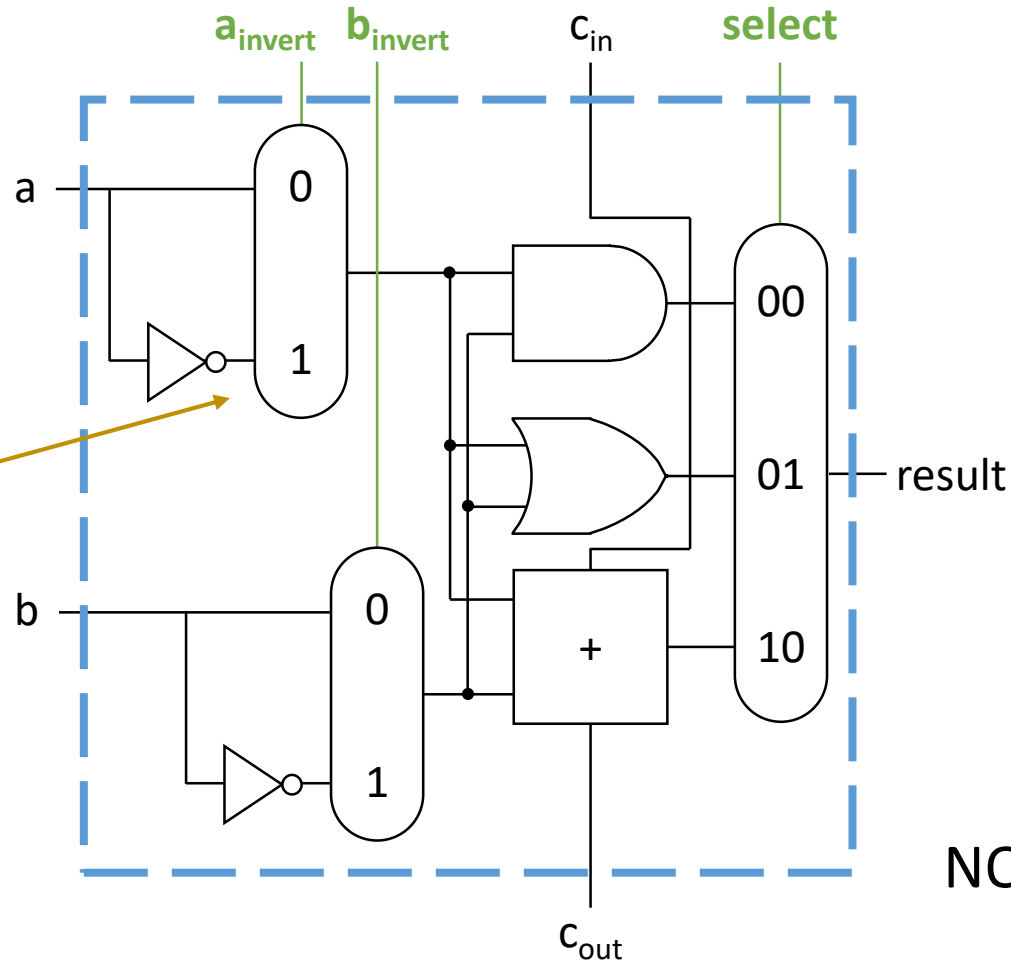
$$\text{NAND: } \overline{a \cdot b} = \bar{a} + \bar{b}$$

- Already have AND, OR, and $\bar{b}$. Just need $\bar{a}$.

1-bit ALU with NOR/NAND

select $\bar{a}$ and $\bar{b}$ if
NOR or NAND

add mux and
inverter



$$\text{NOR: } \overline{a + b} = \bar{a} \cdot \bar{b}$$

$$\text{NAND: } \overline{a \cdot b} = \bar{a} + \bar{b}$$

Set on Less Than

- The *set on less than* (SLT) operation returns 1 if $a < b$ and 0 otherwise
- Implement SLT by subtracting the two operands and checking if the outcome is negative

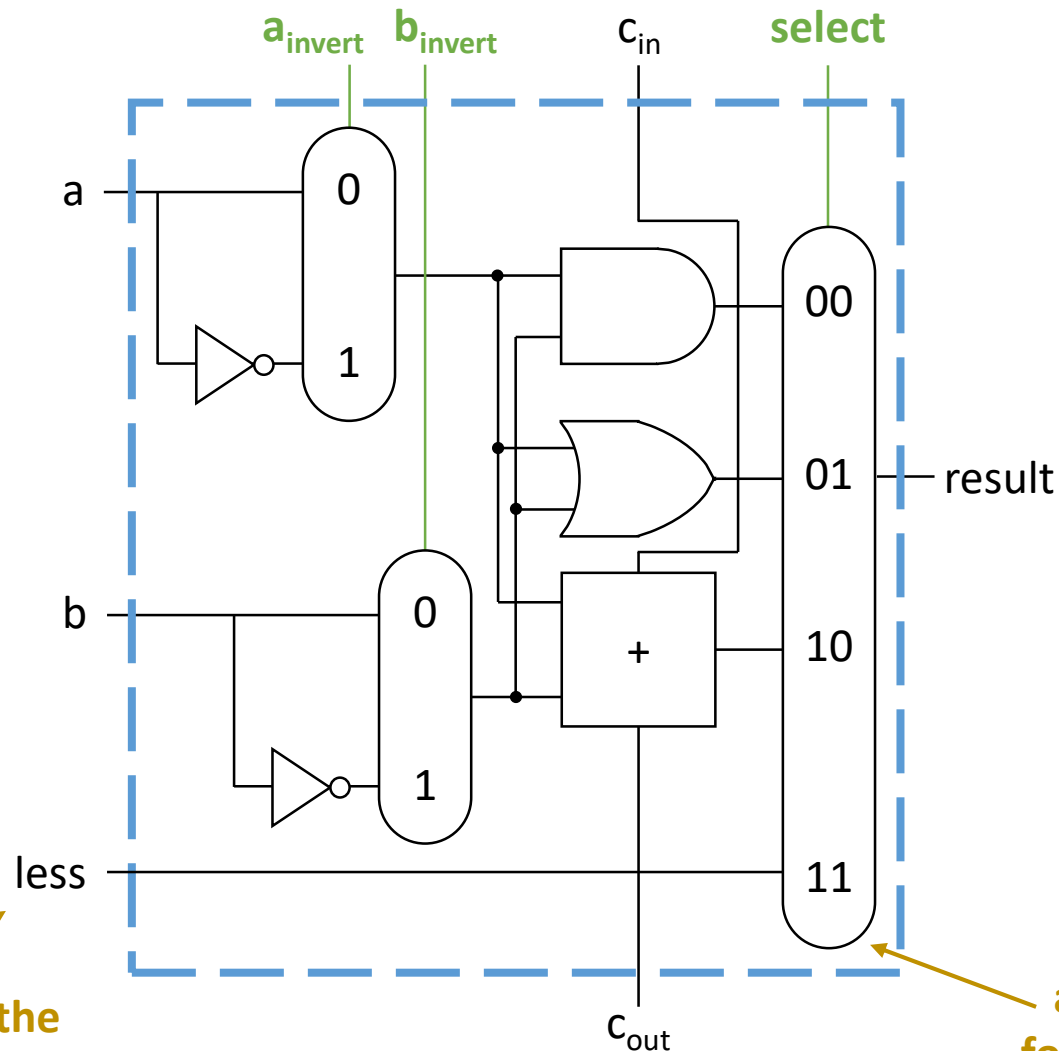
$$a < b \Leftrightarrow a - b < 0$$

- The subtraction outcome is negative if its most significant bit is 1 (property of two's complement)

$$a - b < 0 \Leftrightarrow MSB(a - b) = 1$$

- Result is:
 - 00...00 if $MSB(a - b) = 0$
 - 00...01 if $MSB(a - b) = 1$
$$\longrightarrow \begin{array}{l} result_0 = MSB(a - b) \\ result_i = 0 \text{ for } i > 0 \end{array}$$

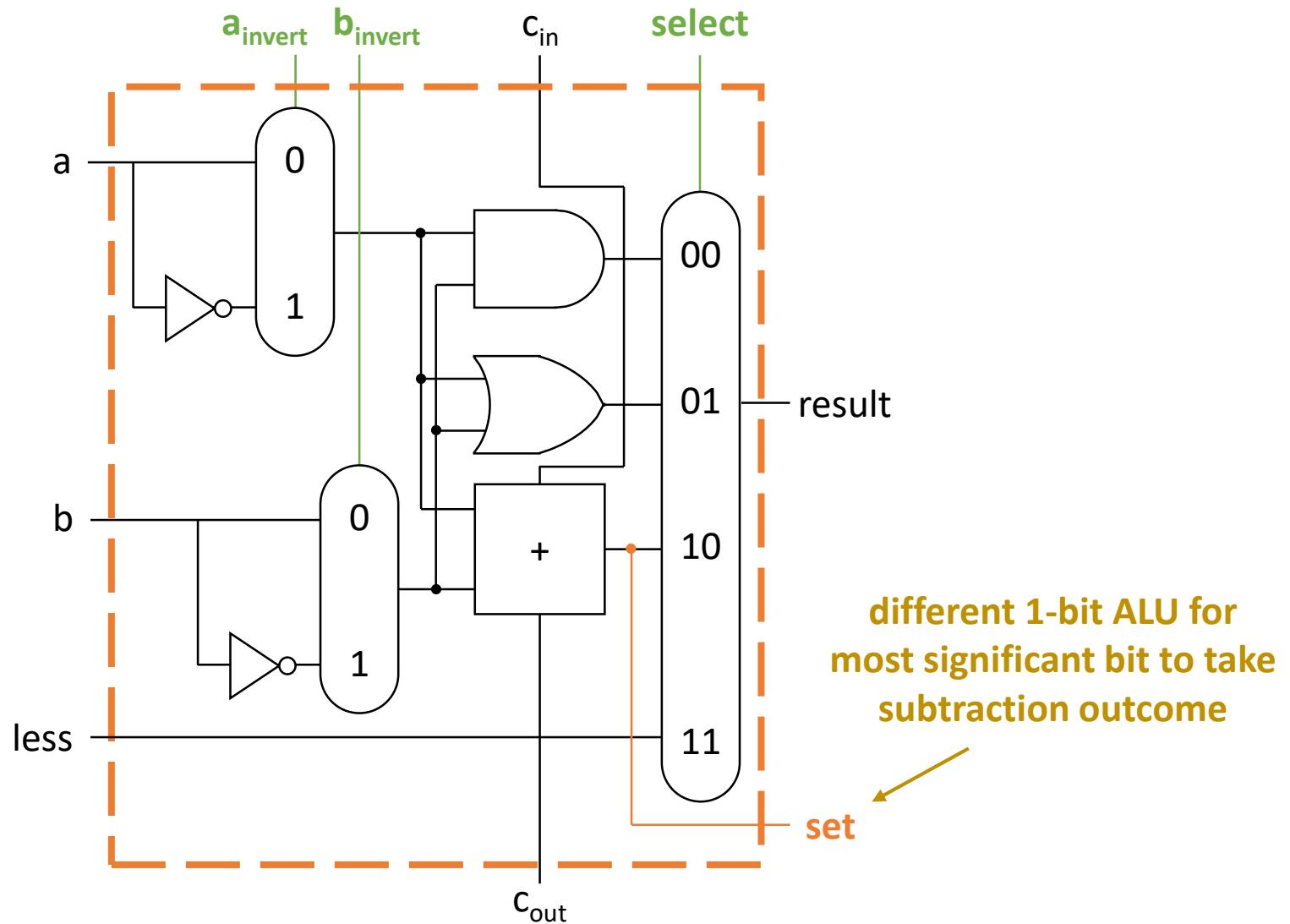
1-bit ALU with SLT



all 0 except for the
least significant bit

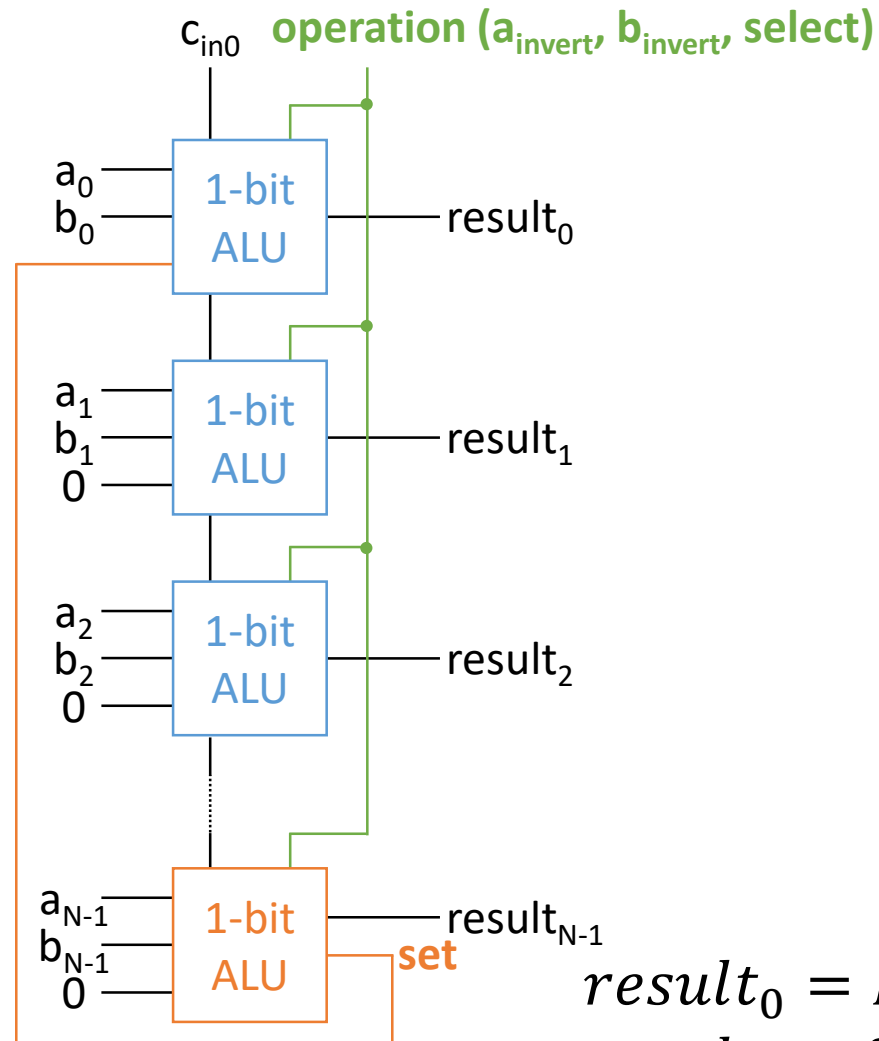
add SLT result as
fourth input to mux

1-bit ALU for MSB with SLT



N-bit ALU with SLT

pass most significant bit
of subtraction outcome
as the *less* value of the
least significant bit



$$result_0 = MSB(a - b)$$
$$result_i = 0 \text{ for } i > 0$$

Textbook Sections

- Some of the content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - B.5